

Assignment 2

Advanced NLP

Monsoon 2026

Deadline. 4th October 2026, 23:59

General Instructions

1. The assignment must be implemented in Python using the PyTorch framework. Other libraries like `einops` or `einx` are permitted as long as `torch` remains the primary framework. You will also be using `huggingface` and related libraries to import checkpoints and datasets.
2. You will be provided a repository on the `code.iiit.ac.in` domain to push your code into. This domain has a very limited storage quota so make sure to not push large files like models or logfiles. You will not be required to upload anything on Moodle. This repository will be archived at the deadline and will be used for evaluations.
3. If you use LLM-based coding agents, please do use the provided `AGENTS.md` or `CLAUDE.md` file included in the repository. While this isn't enforceable, it is highly recommended, as it'll help you make the most of the assignment. You **may not** use LLMs to write or to assist in the writeup in any way.
4. You will only be graded on the key ideas in this assignment (i.e. Mixture of Experts, Optimizers and Decoding strategies). As a thumb rule, you are free to borrow code from anywhere (including your own submission for Assignment 1), as long as **it does not trivially solve the assignment** (or otherwise defeat the purpose of the assignment). For example, you can use `torch.nn.utils.scaled_dot_product_attention` or `tokenizers` library from HuggingFace for the entire assignment. However, you cannot use `torch.nn.optim` for part 2 of the assignment, or `GPTNeoXForCausalLM.generate()` from HuggingFace's `transformers` for part 3 of the assignment. Any attempts to use third-party code to circumvent the purpose of the assignment in any way will be penalised.
5. Upload all model checkpoints to Huggingface, and include a link to them in the README, the writeup, and wherever else required. Ensure that the links are public and accessible to TAs **at the time of the submission**.
6. Upload all model logs to WandB, and include a link to the runs in the README, the writeup and wherever else required. Ensure that the links are public and accessible to TAs **at the time of the submission**. We recommend not deleting any runs, and instead using appropriate tags and notes wherever applicable.
7. Note that there are three tasks in this assignment.

1 Mixture of Experts (40)

The goal of this section is to implement a Mixture-of-Experts (MoE) module that can be used as a drop-in replacement for an MLP-based feed-forward network in a transformer.

You would be required to implement a decoder-only transformer model (trained on the dataset described below) whose feedforward layers are of the five kinds described in the deliverable. The attention module can implement any method you like, feel free to use your implementations from Assignment 1. One (recommended) way to have the FFN modules reusable is described in a code listing in the Appendix. While adding MoE layers to your model, please keep the number of **total** parameters the same for the first 4 ablations. For the last ablation, please keep the number of **active** parameters the same as the dense baseline. This will allow you to compare the result of the

ablating both the total and active number of parameters on the output perplexity and generations. Please keep the number of tokens that each model will be trained on constant, to allow for fair comparison between techniques¹.

You will train your model to translate Vietnamese and Japanese texts to English using samples from [belumind/en-vi-ja-curated-500k-triplets](#). Note that the input will not be romanized so you will have to keep that in mind when training your tokenizers. While there are ~500k entries in the dataset, each such entry will create two training samples for your model since it creates a Vietnamese-English pair and a Japanese-English pair.

Once trained, you will be required to analyse the translation performance of each configuration using perplexity over the test set, as well as the BLEU score [2] as a metric. You will also be required to present a heatmap of the experts that get activated for each input language. This should give an idea of which experts “specialized” on which language.

Low Compute Tip

While we avoid prescribing specific hyperparameters so that you can try out different methods of sweeping over hyperparameters, this assignment requires multiple models to be trained which will likely take multiple attempts each (accounting for debugging). As such, we’d recommend restricting your model to between 10M and 100M parameters. In testing, we found that a ~30M parameter model takes about 1 hour to train on 30M tokens on the dataset on a single 2080 Ti. We expect you to take upto 3 hours for the final runs of each of the ablations, and we request you to take an appropriate amount of time into consideration while planning your assignment.

Deliverable

For this section, you will be required to submit the following:

- An implementation of a transformer model that uses these five FFN variants (one of which is not an MoE approach):
 1. A standard 2-layer MLP.
 2. A MoE layer with 4 experts, with 1 active at a time.
 3. A MoE layer with 4 experts, with 2 active at a time.
 4. A MoE layer with 1 shared expert which is active always, and 3 routed experts, of which 1 will be active at a time (making it a total of 2 out of 4).
 5. A MoE layer with 4 experts, with 2 active at a time, having the same number of active parameters as (1).
- For each model, the trained model checkpoints on HuggingFace and training run logs on WandB.
- Plots and analysis justifying the change of performance by the overall model (or lack thereof) using the different FFN variants on the task provided.
- Analysis of the usage of each expert in the MoE variants to check the distribution of labour (so to speak).

2 Optimizers (30)

For this section, you’ll be using a decoder-only transformer with a dense feedforward network (the same as the first configuration of the previous task) and training it afresh on a new dataset, but this time using different optimizers.

The paper *Fantastic Pretraining Optimizers and Where to Find Them* [3] analyses and benchmarks several optimizers. It has a list of optimizers in its Table 1, with implementation pseudocode in an appendix. The table also classifies the optimizers into five categories. You’ll be required to implement one optimizer of your choice from each of the first 4 categories (i.e. AdamW, Variance-reduced AdamW Variants, Memory-efficient Optimizers, Matrix-based Optimizers). You may attempt Sophia as a bonus. You may not use any predefined `torch.optim` modules, except `torch.optim.Optimizer` (strictly as a base class).

¹A fairer way of measuring this would be keeping FLOPs/token and token counts constant, but we simplify the latter bit for now

To compare each of these implementations, you will be pretraining on [browndw/human-ai-parallel-corpus](#) for next-token prediction. At regular intervals during the training process, compare the training dynamics of each of the models similarly to Figure 2 in Section 4 of the paper. Specifically, prepare and analyse plots of validation loss and test BLEU score with respect to the number of tokens in the dataset. The paper uses units of the Chinchilla ratio [1], and while it explores up to $4\times$ Chinchilla, it might be a bit excessive for the scope of the assignment, so you'll be required to explore the training behaviour up to $1\times$ the dataset (which isn't exactly a clean fraction of the Chinchilla ratio). You can either use the same hyperparameters as the previous task, or perform new sweeps. If you perform new hyperparameter sweeps, please include them in your report.

Low Compute Tip

The model parameters being changed is something you've already addressed in the first task (hopefully), but another possible trick is to use only a portion of the final training regime to debug and test out other (non-model) hyperparameters before evaluating the final configuration on the complete training regime.

Deliverable

For this section, you will be required to submit:

- Implementations of one optimizer from each of the four categories listed above.
- (Bonus) Implementation of the Hessian-based optimizer
- Plots and analysis of validation loss and test BLEU versus tokens used, at several intermediate stages (with a gap of $0.1\times$ dataset tokens)
- For each optimizer, the trained model checkpoints on HuggingFace and training run logs on WandB.
- (Bonus) Hyperparameter Sweeps: If you implement your own hyperparameter sweeps, please include the results from that in the report as well as in separate graphs.

3 Decoding Strategies (30)

This section is about comparing three of the most popular strategies by which the generated token is sampled from the probability distribution that a model generates. Naïvely picking the singular token with the highest probability is one possible strategy. However, this is often suboptimal, particularly in cases where the token has a high entropy (i.e. more confusion about what token to go next). Instead, one can sample from the probability distribution that the model generates, which can lead to some form of variety when generating text. It has also been empirically observed that such techniques help in improving the actual generations of the model, even if there is a slight tradeoff in the more statistical metrics.

For the assignment, you'll be required to implement four decoding strategies from scratch: greedy sampling, beam search (at widths 1, 2 and 4), top-k, and top-p. Using these, you need to evaluate the different decoding strategies using different generation parameters. You need to report the perplexity and test accuracy for each configuration. Additionally, you'll be required to provide an analysis of the time taken by the beam search for the different values of beam width for the same test split. Consequently, describe the overall time complexity of the beam search strategy with respect to beam width.

The model you'll be building on will be the pretrained checkpoint of [EleutherAI/Pythia-160M](#). You CANNOT use `model.generate()`. You have to use `model.forward()` in all of your decoding strategies, and implement your decoding strategy on top of that. Please use [hamishivi/ROCStories](#) to evaluate your decoding strategies, with a minimum of 1000 samples used.

Low Compute Tip

Be careful when implementing beam search! It can very easily hog a lot of resources if not done properly or if your hyperparams are too big.

Deliverable

For this section, you will be required to submit:

- Implementations of the four decoding strategies:
 1. Greedy decoding
 2. Top-p decoding
 3. Top-k decoding
 4. Beam search (width 1, 2 and 4)
- Perplexity and accuracy for each of the techniques
- Analysis of the generations of each of the techniques
- Raw generations from each decoding strategy

Submission

You'll be using `uv` as the Python package manager. The `pyproject` file, the `lockfile` and the `version` file will be created automatically by `uv`. You'll be required to push your code to the assigned repository in `code.iiit.ac.in` with the following directory structure:

```
1 .
2 |-- .gitignore (make sure to ignore all large files including model files (.pt or .safetensors) and
    virtual environment)
3 |-- .python-version
4 |-- uv.lock
5 |-- pyproject.toml
6 |-- README.md
7 |-- report.pdf
8 |-- main.py
9 |-- src/
10 |   |-- utils.py (if required, for helper functions)
11 |   |-- part1/
12 |   |-- part2/
13 |   |-- part3/
14 |   ... (or other suitable structure that ensures modularity and readability)
15 |-- configs/ (if required, yaml-based config files for quick hyperparameter tweaking)
16 |-- assets/
17 |-- ... (if required, any small auxiliary files)
```

Appendix

```
1 import torch.nn as nn
2
3
4 @dataclass
5 class TransformerConfig:
6     D_model: int # Dimension of the model
7     N_ctx: int # Number of context tokens
8     use_moe: bool # Whether to use MoE or MLP
9     ... # et cetera
10
11
12 class MoE(nn.Module):
13     def __init__(self, config: TransformerConfig):
14         super().__init__()
15         ...
16
17     def forward(self, x):
18         # Input shape: (B, N_ctx, D_model)
19         # Output shape: (B, N_ctx, D_model)
```

```

20     ...
21
22
23
24 class MLP(nn.Module):
25     def __init__(self, config: TransformerConfig):
26         super().__init__()
27         ...
28
29     def forward(self, x):
30         # Input shape: (B, N_ctx, D_model)
31         # Output shape: (B, N_ctx, D_model)
32         ...
33
34
35 class Transformer(nn.Module):
36     def __init__(self, config: TransformerConfig):
37         super().__init__()
38         self.ffn = (MoE(config) if config.use_moe else MLP(config))
39         ...
40
41     def forward(self, x):
42         # Input shape: (B, N_ctx, N_vocab)
43         # Output shape: (B, N_ctx, N_vocab)
44         ...

```

Listing 1: Code snippet with drop-in MoE module.

References

- [1] Jordan Hoffmann et al. *Training Compute-Optimal Large Language Models*. 2022. arXiv: [2203.15556](https://arxiv.org/abs/2203.15556) [cs.CL]. URL: <https://arxiv.org/abs/2203.15556>.
- [2] Kishore Papineni et al. “BLEU: a method for automatic evaluation of machine translation”. In: *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*. ACL ’02. Philadelphia, Pennsylvania: Association for Computational Linguistics, 2002, pp. 311–318. DOI: [10.3115/1073083.1073135](https://doi.org/10.3115/1073083.1073135). URL: <https://doi.org/10.3115/1073083.1073135>.
- [3] Kaiyue Wen et al. *Fantastic Pretraining Optimizers and Where to Find Them*. 2025. arXiv: [2509.02046](https://arxiv.org/abs/2509.02046) [cs.LG]. URL: <https://arxiv.org/abs/2509.02046>.